

Tugas 10: K-Nearest Neighbours (KNN)

Oryza Ayunda Putri

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: nasi.tektekmangudin@gmail.com

1. Praktikum 1

Latihan 1

Praktikum

Contoh berikut diambil dari buku "Data Science Algorithms in a Week" (oleh Dávid Natingga). Akan dilakukan klasifikasi suhu udara berdasarkan persepsi seseorang yang bernama Marry. Adapun klasifikasi suhu udara terdiri dari 2 persepsi yaitu **Panas** dan **Dingin**. Persepsi ini dapat diukur berdasarkan 2 variabel yaitu **temperatur dalam derajat celcius** dan **kecepatan angin dalam km/h**.

	Temperatur Udara (°C)	Kecepatan Angin (km/jam)	Klasifikasi atau Persepsi Marry
Data Set (raining)	10	0	Dingin
	25	0	Panas
	15	5	Dingin
	20	3	Panas
	18	7	Dingin
	20	10	Dingin
	22	5	Panas
	24	6	Panas

- Data Test
- Bagaimana persepsi Marry saat temperatur udara 16°C dan kecepatan angin 3 km/jam ?
 - Tentukan nilai k yang tepat berdasarkan akurasi yang paling baik

Dari soal diatas didapatkan hasil perhitungan oleh saya pribadi sebagai berikut:

15	5	dingin	1	4	2.236
18	7	dingin	4	16	4.472
20	3	panas	16	0	4
22	5	panas	36	4	6.325
10	0	dingin	36	9	6.708
20	10	dingin	16	49	8.062
24	6	panas	64	9	8.544
25	0	panas	81	9	9.487

- $= (A3 - H\$3)^2 \rightarrow$ Rumus ini digunakan untuk mencari jarak 15 derajat celcius ke 16 derajat celcius dan akan dipakai ke kolom berikutnya.
- $= (B3 - I\$3)^2 \rightarrow$ Rumus ini juga digunakan untuk menghitung jarak 4 km/jam ke 3 km/jam dan akan dipakai ke kolom berikutnya.
- $= \text{SQRT}(E3 + D3) \rightarrow$ Rumus ini digunakan untuk membagi hasil jarak celcius dengan hasil jarak kecepatan.
- Didapatkan hasil kesimpulan sebagai berikut:

Dingin	3	Maka suhu 16 derajat dengan waktu tempuh
Panas	2	3 km menghasilkan DINGIN
K	5	Kalo hasilnya sama, maka harus ambil data ganjil

2. Praktikum 2

1. Import Library

```
# Step 1: import library & data
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

✓ 0.0s

Pada tahap pertama, dilakukan proses import beberapa library yang diperlukan untuk analisis data dan evaluasi model. Library pandas digunakan untuk mengelola dan memanipulasi data dalam bentuk tabel (DataFrame). Library matplotlib.pyplot dan seaborn dimanfaatkan untuk membuat visualisasi, termasuk grafik heatmap dari confusion matrix. Sementara itu, fungsi-fungsi dari sklearn.metrics seperti confusion_matrix, accuracy_score, precision_score, dan recall_score digunakan untuk mengevaluasi performa model dengan menghitung metrik-metrik evaluasi. Setelah library diimport, data aktual dan data prediksi dimasukkan secara manual sesuai dengan tabel pada soal, sehingga dapat dianalisis lebih lanjut tanpa perlu membaca file eksternal seperti CSV.

2. Membuat Tabel Data Sesuai Soal

Membuat tabel data sesuai soal

```
# Data (dari soal)
actual = [
    "lulus", "lulus", "lulus", "lulus", "lulus",
    "Tidak lulus", "Tidak lulus", "Tidak lulus", "Tidak lulus", "Tidak lulus"
]

predicted = [
    "lulus", "lulus", "lulus", "Tidak lulus", "Tidak lulus",
    "lulus", "Tidak lulus", "Tidak lulus", "Tidak lulus", "Tidak lulus"
]

# Optional: DataFrame supaya mudah dicek
df = pd.DataFrame({"Actual": actual, "Predicted": predicted})
df.head()
```

✓ 0.0s

	Actual	Predicted
0	Lulus	Lulus
1	Lulus	Lulus
2	Lulus	Lulus
3	Lulus	Tidak Lulus
4	Lulus	Tidak Lulus

Python

Pada tahap ini, data dimasukkan secara manual berdasarkan tabel hasil prediksi yang diberikan pada soal. Data terdiri dari dua komponen, yaitu actual (hasil sebenarnya) dan predicted (hasil prediksi model). Kedua data tersebut disimpan dalam bentuk list agar dapat diproses oleh fungsi evaluasi model pada tahap selanjutnya. Untuk memudahkan pengecekan dan memastikan bahwa data telah masuk dengan benar, kedua list ini kemudian digabungkan ke dalam sebuah DataFrame menggunakan library pandas. DataFrame ini menampilkan kolom "Actual" dan "Predicted" sehingga mempermudah

proses validasi sebelum dilakukan perhitungan confusion matrix dan metrik evaluasi lainnya.

3. Array untuk Confusion Matrix

```
array untuk confusion matrix

# Step 2: confusion matrix (array)
labels = ["Lulus", "Tidak Lulus"] # urutan label yang konsisten
cm = confusion_matrix(actual, predicted, labels=labels)
cm

✓ 0.0s

array([[3, 2],
       [1, 4]])
```

Pada tahap ini, dilakukan perhitungan confusion matrix untuk mengevaluasi kinerja model berdasarkan data aktual dan data prediksi yang telah dimasukkan sebelumnya. Confusion matrix disusun dalam bentuk array menggunakan fungsi `confusion_matrix` dari library `sklearn.metrics`. Agar hasil perhitungan lebih konsisten dan sesuai dengan urutan kelas yang diinginkan, ditetapkan label secara manual dalam variabel `labels`, yaitu “Lulus” dan “Tidak Lulus”. Penentuan urutan label ini penting untuk memastikan bahwa posisi True Positive, False Negative, False Positive, dan True Negative terbaca dengan benar pada kolom dan baris matriks. Hasil akhir berupa array 2×2 yang merepresentasikan jumlah prediksi benar maupun salah pada masing-masing kelas.

4. Menghitung Accuracy, Precision, Recall

```
# Step 3: hitung metrik
accuracy = accuracy_score(actual, predicted)
precision = precision_score(actual, predicted, pos_label="Lulus")
recall = recall_score(actual, predicted, pos_label="Lulus")

print(f"Accuracy : {accuracy:.2f} ({accuracy*100:.1f}%)")
print(f"Precision: {precision:.2f} ({precision*100:.1f}%)")
print(f"Recall   : {recall:.2f} ({recall*100:.1f}%)")

✓ 0.0s

Accuracy : 0.70 (70.0%)
Precision: 0.75 (75.0%)
Recall   : 0.60 (60.0%)
```

Pada tahap ini dilakukan perhitungan metrik evaluasi untuk mengukur performa model prediksi berdasarkan hasil confusion matrix. Tiga metrik utama yang dihitung adalah accuracy, precision, dan recall. Metrik accuracy digunakan untuk mengetahui persentase keseluruhan prediksi yang sesuai dengan data aktual. Precision dihitung untuk melihat tingkat ketepatan model dalam memprediksi kelas “Lulus”, dengan membandingkan jumlah prediksi benar pada kelas tersebut terhadap total prediksi “Lulus”. Sedangkan recall mengukur sejauh mana model mampu mengenali seluruh data aktual yang termasuk dalam kelas “Lulus”. Masing-masing metrik dihitung menggunakan fungsi yang disediakan oleh `sklearn.metrics`, yaitu `accuracy_score`, `precision_score`, dan `recall_score`. Hasil perhitungan kemudian ditampilkan dalam bentuk persentase agar lebih mudah dipahami dan dianalisis.

5. Visualisasi

```
plt.figure(figsize=(6,4))

# tanpa pilih warna manual (sesuai aturan matplotlib)
plt.imshow(cm)

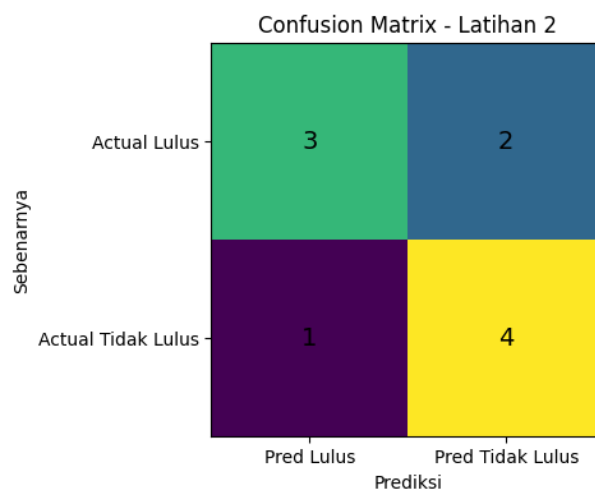
# Tampilkan angka
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j], ha='center', va='center', fontsize=14)

# Label sumbu
plt.xticks([0,1], ["Pred Lulus", "Pred Tidak Lulus"])
plt.yticks([0,1], ["Actual Lulus", "Actual Tidak Lulus"])

plt.title("Confusion Matrix - Latihan 2")
plt.xlabel("Prediksi")
plt.ylabel("Sebenarnya")

plt.show()
```

Pada tahap ini, confusion matrix yang telah dihitung sebelumnya divisualisasikan menggunakan grafik berbasis warna untuk memudahkan interpretasi. Visualisasi dibuat dengan fungsi `plt.imshow(cm)` dari *matplotlib*, tanpa menentukan warna secara manual sehingga tetap mengikuti konfigurasi default yang diizinkan. Setiap sel dalam matriks kemudian diberi anotasi berupa angka yang menunjukkan jumlah prediksi pada kombinasi kelas tertentu, sehingga informasi yang ditampilkan menjadi lebih jelas dan mudah dibaca. Label pada sumbu X dan Y ditambahkan untuk menunjukkan kategori prediksi dan hasil sebenarnya, yaitu “Lulus” dan “Tidak Lulus”. Judul dan label sumbu turut disertakan agar grafik lebih informatif dan sesuai dengan format visualisasi pada laporan praktikum. Dengan adanya visualisasi ini, pembaca dapat dengan cepat memahami pola kesalahan dan keberhasilan model dalam melakukan klasifikasi.



3. Praktikum 3

1. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.decomposition import PCA
from imblearn.over_sampling import SMOTE

df = pd.read_csv("../data/weather.csv", sep=";")
df.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	inland	Rainy
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	inland	Cloudy
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	mountain	Sunny
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	coastal	Sunny
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	mountain	Rainy

Pada tahap ini dilakukan proses import berbagai library yang diperlukan untuk analisis data, pra-pemrosesan, serta pembangunan model klasifikasi. Library pandas digunakan untuk membaca dan mengelola data dalam bentuk DataFrame, sementara numpy berfungsi untuk operasi numerik. Matplotlib dan seaborn dimanfaatkan untuk membuat visualisasi seperti grafik dan heatmap. Library sklearn menyediakan berbagai fungsi penting, seperti `train_test_split` untuk membagi data menjadi train dan test, `StandardScaler` untuk normalisasi fitur, `GridSearchCV` untuk optimasi parameter KNN, serta `classification_report` dan `confusion_matrix` untuk evaluasi model. Selain itu, SMOTE dari library imblearn digunakan untuk menangani data yang tidak seimbang dengan melakukan oversampling pada kelas minoritas.

Setelah seluruh library diimpor, dataset dibaca menggunakan `pd.read_csv()` dari file `weather.csv`. Dataset ini kemudian ditampilkan menggunakan `df.head()` untuk melihat lima baris pertama, memastikan struktur kolom telah sesuai dan data berhasil dimuat dengan benar sebelum melanjutkan analisis pada tahap berikutnya.

2. Membagi Dataset Menjadi Training dan Testing

```
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
```

Pada langkah ini, dataset dibagi menjadi dua bagian, yaitu *train set* dan *test set*, menggunakan fungsi `train_test_split` dari library `sklearn.model_selection`. Pembagian data ini bertujuan untuk memisahkan data yang akan digunakan untuk melatih model (train) dan data yang digunakan untuk menguji performa model (test). Pada praktikum ini, proporsi data yang digunakan adalah **80% untuk training** dan **20% untuk testing**, yang ditentukan melalui parameter `test_size=0.2`. Selain itu, digunakan parameter `random_state=42` untuk memastikan proses pembagian data bersifat konsisten setiap kali dijalankan, sehingga hasil eksperimen dapat direproduksi secara tepat. Hasil pembagian ini disimpan dalam variabel `train_df` dan `test_df`, yang kemudian menjadi dasar dalam proses pra-pemrosesan dan pelatihan model pada tahap berikutnya.

3. Menyimpan Dataset Hasil Pembagian ke File CSV

```
train_df.to_csv("weather_train.csv", index=False)  
test_df.to_csv("weather_test.csv", index=False)
```

Pada tahap ini, dataset yang telah dibagi menjadi train set dan test set disimpan kembali ke dalam file CSV menggunakan fungsi `to_csv()` dari library pandas. Penyimpanan ini dilakukan agar data hasil pembagian dapat digunakan kembali tanpa perlu mengulangi proses train-test split, sehingga mempermudah proses analisis dan memastikan konsistensi data pada setiap percobaan. File yang dihasilkan adalah `weather_train.csv` untuk data training dan `weather_test.csv` untuk data testing. Parameter `index=False` digunakan untuk menghindari penyimpanan indeks DataFrame ke dalam file, sehingga format data tetap bersih dan konsisten. Langkah ini juga berguna apabila data ingin dipindahkan atau digunakan pada sistem lain tanpa perlu memuat ulang dataset asli.

4. Membaca Kembali Dataset dari Train Csv dan Test Csv

```
train_df= pd.read_csv('../data/weather_test.csv', sep=',')  
train_df.head()  
  
test_df= pd.read_csv('../data/weather_train.csv', sep=',')  
test_df.head()
```

Pada tahap ini, dataset *train* dan *test* yang sebelumnya telah disimpan dalam format CSV dibaca kembali menggunakan fungsi `pd.read_csv()` dari library *pandas*. Langkah ini dilakukan untuk memastikan bahwa file hasil penyimpanan dapat dimuat dengan benar serta dapat digunakan pada proses analisis selanjutnya tanpa ketergantungan pada dataset awal. Parameter `sep=','` digunakan untuk menegaskan bahwa pemisah antar kolom pada file adalah koma, sesuai dengan format CSV standar. Setelah file dibaca, perintah `train_df.head()` digunakan untuk menampilkan lima baris pertama dari dataset guna memverifikasi bahwa struktur data, tipe kolom, serta isi dataset telah terbaca dengan benar. Proses ini merupakan bagian penting dalam memastikan integritas data sebelum melanjutkan ke tahap pra-pemrosesan atau pemodelan.

5. Pemeriksaan Struktur Dataset

```
train_df.info()  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2640 entries, 0 to 2639  
Data columns (total 11 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   Temperature            2640 non-null   float64  
1   Humidity                2640 non-null   int64  
2   Wind Speed             2640 non-null   float64  
3   Precipitation (%)       2640 non-null   float64  
4   Cloud Cover            2640 non-null   object  
5   Atmospheric Pressure    2640 non-null   float64  
6   UV Index               2640 non-null   int64  
7   Season                 2640 non-null   object  
8   Visibility (km)         2640 non-null   float64  
9   Location                2640 non-null   object  
10  Weather Type           2640 non-null   object  
dtypes: float64(5), int64(2), object(4)  
memory usage: 227.0+ KB
```

```
test_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10560 entries, 0 to 10559
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Temperature            10560 non-null  float64
1   Humidity                10560 non-null  int64  
2   Wind Speed             10560 non-null  float64
3   Precipitation (%)      10560 non-null  float64
4   Cloud Cover            10560 non-null  object  
5   Atmospheric Pressure   10560 non-null  float64
6   UV Index               10560 non-null  int64  
7   Season                 10560 non-null  object  
8   Visibility (km)        10560 non-null  float64
9   Location                10560 non-null  object  
10  Weather Type           10560 non-null  object  
dtypes: float64(5), int64(2), object(4)
memory usage: 907.6+ KB
```

Pada langkah ini dilakukan pemeriksaan struktur dataset menggunakan perintah `train_df.info()` dan `test_df.info()`. Fungsi ini menampilkan informasi penting mengenai dataset, termasuk jumlah baris dan kolom, nama setiap kolom, tipe data masing-masing atribut, serta jumlah nilai non-null yang terdapat pada setiap kolom. Pemeriksaan ini bertujuan untuk memastikan bahwa dataset telah terbaca dengan benar, tidak terdapat kolom yang hilang, serta mengetahui apakah terdapat *missing values* yang harus ditangani pada tahap pra-pemrosesan berikutnya. Selain itu, informasi mengenai tipe data juga membantu dalam menentukan metode transformasi atau encoding yang sesuai pada tahap analisis berikutnya. Dengan demikian, langkah ini menjadi fondasi penting sebelum melanjutkan ke proses pembersihan dan persiapan data.

6. Pemeriksaan Nilai Unik Pada Atribut Kategorikal

```
print(train_df['Cloud Cover'].unique())
print(train_df['Season'].unique())
print(train_df['Location'].unique())
print(train_df['Weather Type'].unique())

['clear' 'partly cloudy' 'overcast' 'cloudy']
['Autumn' 'Summer' 'Spring' 'Winter']
['inland' 'mountain' 'coastal']
['Sunny' 'Snowy' 'Rainy' 'Cloudy']
```

Pada tahap ini dilakukan pemeriksaan nilai unik untuk beberapa atribut kategorikal dalam dataset, yaitu *Cloud Cover*, *Season*, *Location*, dan *Weather Type*. Pemeriksaan dilakukan menggunakan fungsi `.unique()` pada setiap kolom. Langkah ini bertujuan untuk mengetahui variasi kategori yang terdapat pada masing-masing atribut, sehingga dapat membantu dalam menentukan teknik encoding yang tepat pada tahap pra-pemrosesan data. Selain itu, pengecekan ini memastikan bahwa tidak terdapat kategori yang tidak wajar, salah tulis (*typo*), atau kelas yang kosong. Informasi nilai unik juga memudahkan penyesuaian label untuk proses konversi data menjadi bentuk numerik agar dapat digunakan pada algoritma KNN yang membutuhkan input berupa nilai numerik.

7. Melakukan Encoding pada Atribut Kategorikal

```
train_df['Cloud Cover'] = train_df['Cloud Cover'].replace(
    {'clear': 0, 'partly cloudy': 1, 'cloudy': 2, 'overcast': 3}
)

train_df['Season'] = train_df['Season'].replace(
    {'Winter': 0, 'Spring': 1, 'Summer': 2, 'Autumn': 3}
)

train_df['Location'] = train_df['Location'].replace(
    {'inland': 0, 'coastal': 1, 'mountain': 2}
)

train_df['Weather Type'] = train_df['Weather Type'].replace(
    {'Sunny': 0, 'Cloudy': 1, 'Rainy': 2, 'Snowy': 3}
)
```

Python

C:\Users\lenovo\AppData\Local\Temp\ipykernel_6220\110830121.py:1: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version of pandas.
train_df['Cloud Cover'] = train_df['Cloud Cover'].replace(
C:\Users\lenovo\AppData\Local\Temp\ipykernel_6220\110830121.py:5: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version of pandas.
train_df['Season'] = train_df['Season'].replace(
C:\Users\lenovo\AppData\Local\Temp\ipykernel_6220\110830121.py:9: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version of pandas.
train_df['Location'] = train_df['Location'].replace(
C:\Users\lenovo\AppData\Local\Temp\ipykernel_6220\110830121.py:13: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version of pandas.
train_df['Weather Type'] = train_df['Weather Type'].replace(

Pada tahap ini dilakukan proses encoding untuk mengubah nilai pada atribut kategorikal menjadi format numerik. Hal ini diperlukan karena algoritma KNN tidak dapat memproses data berbasis teks secara langsung. Proses konversi dilakukan menggunakan fungsi `.replace()` untuk mengganti setiap kategori dengan nilai numerik yang telah ditentukan. Atribut Cloud Cover dikonversi menjadi nilai 0 hingga 3 berdasarkan tingkat kondisi awan. Atribut Season diubah mengikuti urutan musim Winter hingga Autumn, sedangkan Location dikonversi sesuai jenis lokasi seperti inland, coastal, dan mountain. Pada atribut Weather Type, setiap kategori cuaca seperti Sunny, Cloudy, Rainy, dan Snowy diberikan label angka 0 hingga 3. Dengan adanya encoding ini, seluruh atribut kategorikal dalam dataset telah siap digunakan dalam proses pelatihan model KNN.

8. Membuat Dictionary Mapping untuk Encoding Atribut Kategorikal

```
replacements = {
    'Cloud Cover': {
        'clear': 0,
        'partly cloudy': 1,
        'cloudy': 2,
        'overcast': 3
    },
    'Season': {
        'Winter': 0,
        'Spring': 1,
        'Summer': 2,
        'Autumn': 3
    },
    'Location': {
        'inland': 0,
        'coastal': 1,
        'mountain': 2
    },
    'Weather Type': { # TARGET
        'Sunny': 0,
        'Cloudy': 1,
        'Rainy': 2,
        'Snowy': 3
    }
}
```


Pada langkah ini dibuat sebuah dictionary bernama `replacements` yang berisi pasangan *key-value* untuk mengonversi data kategorikal menjadi nilai numerik. Dictionary ini digunakan sebagai acuan untuk proses encoding pada tahap selanjutnya. Setiap atribut kategorikal seperti *Cloud Cover*, *Season*, *Location*, dan *Weather Type* diberikan struktur mapping yang jelas dan konsisten. Misalnya, kondisi awan seperti “clear”, “partly cloudy”, “cloudy”, dan “overcast” diberi label angka 0 hingga 3. Demikian pula atribut lain seperti musim, jenis lokasi, serta tipe cuaca dikonversi berdasarkan urutan maupun karakteristik kategorinya. Penyusunan dictionary ini bertujuan untuk memudahkan proses penggantian nilai menggunakan metode `.replace()` sehingga encoding dapat dilakukan dengan lebih efisien, terorganisir, dan minim kesalahan.

9. Menerapkan Mapping Encoding pada Dataset Train dan Test

```
train_df = train_df.replace(replacements, inplace=False)
test_df = test_df.replace(replacements, inplace=False)
```

Pada tahap ini dilakukan penerapan proses encoding pada dataset training dan testing menggunakan dictionary `replacements` yang telah didefinisikan sebelumnya. Proses ini dilakukan dengan memanfaatkan fungsi `replace()` dari library `pandas`, sehingga setiap nilai kategorikal pada kolom *Cloud Cover*, *Season*, *Location*, dan *Weather Type* secara otomatis diganti dengan nilai numerik sesuai dengan mapping yang telah ditentukan. Penggunaan parameter `inplace=False` memastikan bahwa `DataFrame` asli tidak dimodifikasi secara langsung, melainkan menghasilkan `DataFrame` baru yang telah ter-encode dan kemudian disimpan kembali ke variabel `train_df` dan `test_df`. Langkah ini penting agar keseluruhan dataset memiliki format numerik yang kompatibel untuk digunakan pada tahap selanjutnya, terutama proses normalisasi dan pelatihan model KNN.

10. Pengecekan Missing Value pada Dataset Train

```
train_df.isnull().sum()
```

Temperature	0
Humidity	0
Wind Speed	0
Precipitation (%)	0
Cloud Cover	0
Atmospheric Pressure	0
UV Index	0
Season	0
Visibility (km)	0
Location	0
Weather Type	0
dtype: int64	

```
test_df.isnull().sum()
```

Temperature	0
Humidity	0
Wind Speed	0
Precipitation (%)	0
Cloud Cover	0
Atmospheric Pressure	0
UV Index	0
Season	0
Visibility (km)	0
Location	0
Weather Type	0
dtype: int64	

Pada langkah ini dilakukan pengecekan adanya nilai kosong (*missing values*) dalam dataset training menggunakan fungsi `train_df.isnull().sum()`. Perintah ini menghitung jumlah nilai kosong pada setiap kolom yang terdapat dalam `DataFrame`. Tahap ini sangat penting dilakukan sebelum proses pelatihan model, karena keberadaan nilai kosong dapat mengganggu proses perhitungan jarak pada algoritma KNN dan menyebabkan

model tidak dapat berjalan dengan optimal. Dengan mengetahui kolom mana yang memiliki nilai hilang, peneliti dapat menentukan langkah penanganan yang tepat, seperti menghapus baris yang tidak lengkap atau melakukan imputasi nilai. Hasil dari pengecekan ini menjadi dasar untuk memastikan bahwa dataset dalam kondisi bersih dan siap digunakan pada tahap pemrosesan selanjutnya.

11. Menghapus Missing Value dari Dataset Train dan Test

```
df_train = train_df.dropna()
test_df = test_df.dropna()
```

Pada tahap ini dilakukan proses pembersihan data dengan menghapus seluruh baris yang mengandung nilai kosong (missing values) menggunakan fungsi `dropna()`. Perintah `df_train = train_df.dropna()` dan `test_df = test_df.dropna()` memastikan bahwa hanya data yang lengkap dan valid yang digunakan dalam proses pelatihan dan pengujian model. Langkah ini penting karena algoritma KNN tidak dapat memproses data dengan nilai kosong, terutama karena perhitungan jarak antar data membutuhkan nilai numerik lengkap pada setiap fitur. Dengan menghilangkan baris yang tidak memiliki informasi penuh, kualitas data menjadi lebih baik dan potensi kesalahan selama proses modeling dapat diminimalkan. Setelah tahap ini, dataset dipastikan dalam kondisi bersih dan siap untuk dilanjutkan ke proses normalisasi dan pelatihan model KNN.

12. Menghapus Kolom yang Tidak Digunakan dalam Analisis Korelasi

```
# Kolom yang ingin digunakan untuk korelasi
train_df = train_df.drop(columns=['Visibility (km)'])
train_df.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Location	Weather Type
0	23.0	38	4.5	6.0	0	1021.19	9	3	0	0
1	62.0	94	14.5	83.0	0	1025.57	10	2	0	0
2	40.0	51	2.0	12.0	1	1025.15	8	1	2	0
3	-7.0	79	6.5	90.0	3	993.56	0	0	0	3
4	21.0	94	10.0	109.0	1	1027.71	11	1	2	0

```
test_df = test_df.drop(columns=['Visibility (km)'])
test_df.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Location	Weather Type
0	32.0	66	6.0	92.0	1	1010.21	2	1	0	2
1	15.0	96	11.0	57.0	3	1016.98	0	3	2	2
2	28.0	58	6.5	11.0	0	1025.37	10	2	0	0
3	47.0	84	44.5	85.0	3	992.26	2	0	1	2
4	18.0	66	14.0	10.0	1	1000.04	1	3	1	1

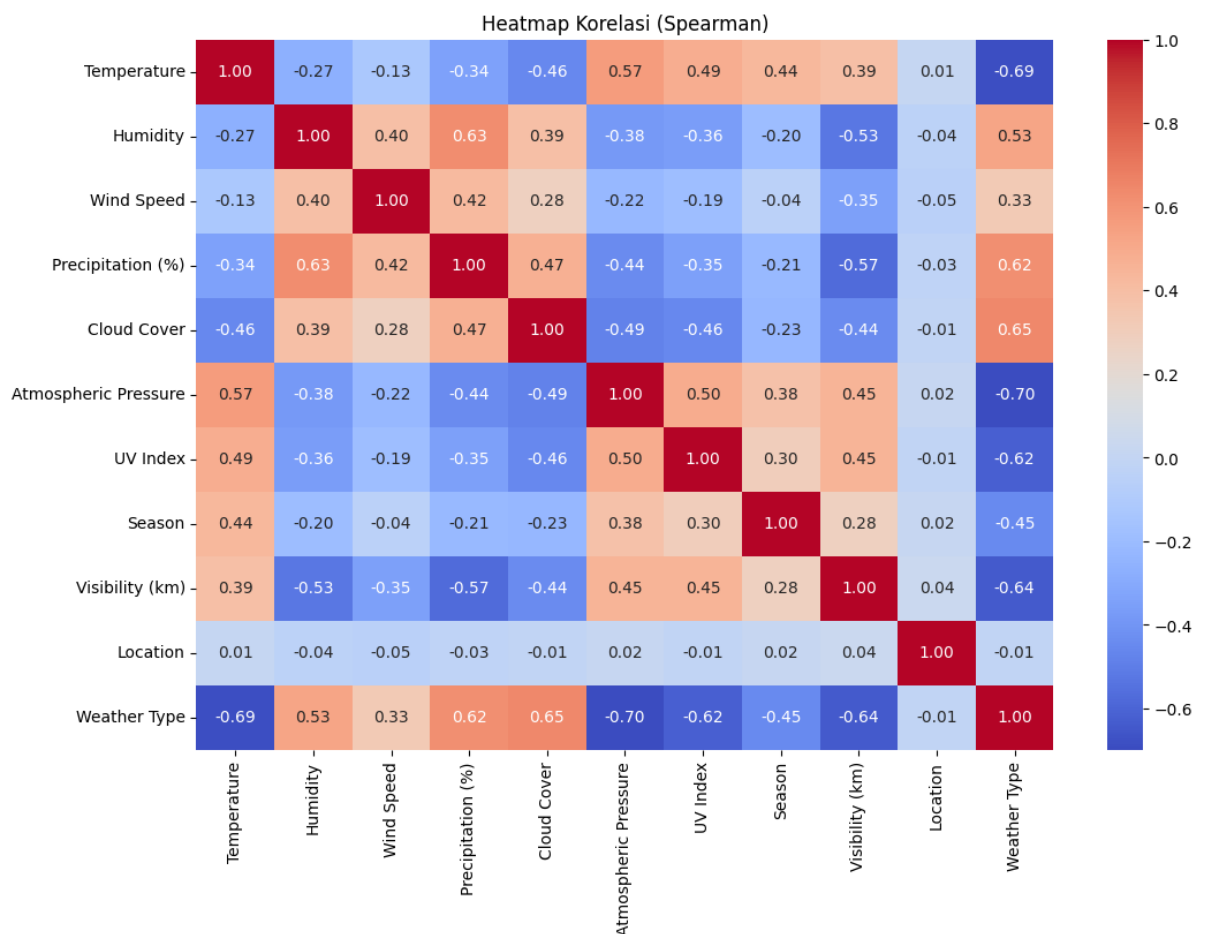
Pada tahap ini dilakukan penghapusan kolom *Visibility (km)* dari dataset training menggunakan fungsi `drop()`. Penghapusan kolom dilakukan karena atribut tersebut dianggap tidak relevan atau tidak dibutuhkan dalam analisis korelasi maupun pemodelan KNN yang akan dilakukan. Dengan menghapus fitur yang kurang berkontribusi, proses analisis data dapat menjadi lebih fokus dan efisien. Selain itu, pengurangan jumlah fitur juga dapat membantu mengurangi kompleksitas perhitungan jarak pada algoritma KNN. Setelah kolom dihapus, dataset diperiksa kembali menggunakan `head()` untuk memastikan bahwa perubahan telah diterapkan dengan benar dan struktur data tetap konsisten.

13. Perhitungan Korelasi Spearman dan Visualisasi Heatmap

```
# Menghitung matriks korelasi menggunakan Spearman
correlation_matrix = df_train.corr(method='spearman')

# Membuat heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Heatmap Korelasi (Spearman)")
plt.show()
```

Pada tahap ini dilakukan perhitungan matriks korelasi antar fitur numerik menggunakan metode **Spearman** dengan perintah `df_train.corr(method='spearman')`. Metode Spearman dipilih karena mampu mengukur hubungan monotonik antar variabel, baik linear maupun non-linear, sehingga lebih fleksibel digunakan pada data dengan distribusi tidak normal atau hubungan yang tidak sepenuhnya linier. Hasil perhitungan korelasi kemudian divisualisasikan dalam bentuk **heatmap** menggunakan library *seaborn*. Visualisasi ini menampilkan nilai korelasi dalam bentuk warna dan anotasi angka, sehingga memudahkan dalam mengidentifikasi fitur-fitur yang memiliki hubungan kuat, lemah, atau tidak memiliki korelasi sama sekali. Ukuran grafik diperbesar melalui `figsize` agar informasi dapat terbaca dengan jelas. Tahap ini membantu dalam memahami karakteristik antar fitur dan dapat menjadi dasar pertimbangan dalam pemilihan variabel yang digunakan pada proses pemodelan KNN.



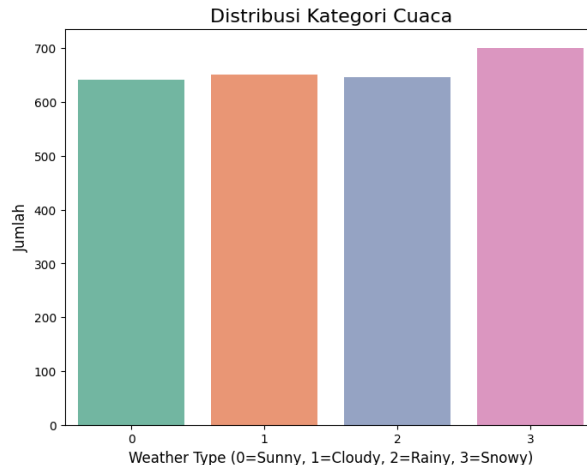
14. Mengecek Distribusi Kelas pada Kolom Weather Type

```
df_train['Weather Type'].value_counts()

Weather Type
3    701
1    651
2    647
0    641
Name: count, dtype: int64
```

Pada tahap ini dilakukan pengecekan distribusi kelas pada kolom Weather Type menggunakan fungsi `value_counts()`. Perintah ini menghitung jumlah kemunculan setiap kategori cuaca yang telah dikonversi menjadi nilai numerik. Langkah ini penting dilakukan untuk mengetahui apakah data target memiliki distribusi yang seimbang atau tidak. Ketidakseimbangan kelas dapat memengaruhi performa model KNN, terutama dalam proses klasifikasi, karena model cenderung bias terhadap kelas mayoritas. Dengan mengetahui distribusi awal kelas, peneliti dapat menentukan apakah diperlukan teknik penyeimbangan data seperti SMOTE pada tahap selanjutnya. Informasi distribusi ini juga membantu dalam memahami karakteristik dataset serta memastikan keakuratan analisis sebelum memasuki tahap modeling.

15. Visualisasi Distribusi Kategori Cuaca Menggunakan Countplot



Pada tahap ini dilakukan visualisasi distribusi kelas pada kolom Weather Type menggunakan fungsi `countplot()` dari library `seaborn`. Visualisasi ini menampilkan jumlah data untuk setiap kategori cuaca yang telah dikonversi ke dalam bentuk numerik (0 = Sunny, 1 = Cloudy, 2 = Rainy, 3 = Snowy). Grafik batang ini bertujuan untuk mempermudah identifikasi apakah terdapat ketidakseimbangan kelas pada data target. Ketidakseimbangan kelas dapat berdampak pada performa algoritma KNN, karena model cenderung lebih akurat pada kelas dengan jumlah data lebih banyak. Dengan menggunakan grafik berwarna dari palet `Set2`, visualisasi menjadi lebih jelas dan mudah dianalisis. Tahap ini merupakan langkah penting sebelum melakukan teknik penyeimbangan seperti SMOTE untuk memastikan bahwa model tidak bias terhadap kelas mayoritas.

16. Pemisahan Fitur dan Label serta Normalisasi Data

```
# Pisahkan fitur & label (train)
x_train = train_df.drop(columns=['Weather Type'])
y_train = train_df['Weather Type']

# Pisahkan fitur & label (test)
x_test = test_df.drop(columns=['Weather Type'])
y_test = test_df['Weather Type']

# Normalisasi
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Pada langkah ini dilakukan pemisahan antara fitur (*independent variables*) dan label (*dependent variable*) pada dataset training dan testing. Kolom *Weather Type* dipisahkan sebagai label (y), sedangkan kolom-kolom lainnya digunakan sebagai fitur (X). Pemisahan ini diperlukan agar model KNN dapat dilatih menggunakan data input (fitur) untuk memprediksi nilai target (label). Setelah proses pemisahan, dilakukan normalisasi menggunakan metode *StandardScaler* untuk menyamakan skala antar fitur. Normalisasi penting dilakukan pada KNN karena algoritma ini bergantung pada perhitungan jarak antar data; fitur yang memiliki skala lebih besar dapat mendominasi perhitungan jarak dan menyebabkan hasil prediksi menjadi bias. Pada tahap ini, *StandardScaler* menghitung nilai rata-rata dan standar deviasi dari data training, kemudian menerapkannya pada kedua dataset, yaitu `x_train_scaled` dan `x_test_scaled`. Dengan demikian, seluruh fitur memiliki skala yang seragam dan siap digunakan untuk proses pelatihan model KNN.

17. Penerapan SMOTE untuk Menyeimbangkan Distribusi Kelas

```
print("Before SMOTE:")
print(y_train.value_counts())

sm = SMOTE(random_state=42, k_neighbors=1)
x_train_smote, y_train_smote = sm.fit_resample(x_train_scaled, y_train)

print("After SMOTE:")
print(y_train_smote.value_counts())
```

```
Before SMOTE:
Weather Type
3    701
1    651
2    647
0    641
Name: count, dtype: int64
After SMOTE:
Weather Type
0    701
3    701
2    701
1    701
Name: count, dtype: int64
```

Pada tahap ini dilakukan proses penyeimbangan kelas menggunakan metode SMOTE (Synthetic Minority Over-sampling Technique). Sebelum SMOTE diterapkan, dilakukan

pengecekan distribusi kelas dengan perintah `y_train.value_counts()`, untuk mengetahui apakah terdapat ketidakseimbangan kelas pada label Weather Type. Ketidakseimbangan ini dapat menyebabkan model KNN cenderung bias terhadap kelas mayoritas. SMOTE bekerja dengan cara membuat sampel sintetis baru pada kelas minoritas berdasarkan kedekatan (k-nearest neighbors) dari data yang sudah ada. Pada praktikum ini digunakan parameter `k_neighbors=1` agar proses oversampling lebih sederhana dan sesuai dengan kondisi dataset. Setelah SMOTE diterapkan, dilakukan pengecekan ulang distribusi kelas untuk memastikan bahwa jumlah data pada setiap kategori telah seimbang. Dataset yang telah diseimbangkan (`x_train_smote` dan `y_train_smote`) selanjutnya digunakan pada proses pelatihan model KNN untuk menghasilkan performa klasifikasi yang lebih optimal.

18. Pemilihan Parameter Terbaik Model KNN dengan Grid Search dan Cross-Validation

```
param_grid = {
    'n_neighbors': list(range(1, 31)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, n_jobs=-1)
grid.fit(x_train_smote, y_train_smote)

print("Best Params:", grid.best_params_)
print("Best Score (CV):", grid.best_score_)

Best Params: {'metric': 'manhattan', 'n_neighbors': 6, 'weights': 'distance'}
Best Score (CV): 0.8866036414565827
```

Pada proses ini, dilakukan optimasi hyperparameter untuk model K-Nearest Neighbors (KNN) menggunakan metode Grid Search dengan validasi silang (cross-validation). Parameter yang diuji meliputi jumlah tetangga terdekat (`n_neighbors`) dari 1 hingga 30, metode pembobotan (`weights`) berupa 'uniform' dan 'distance', serta jenis metrik jarak (`metric`) berupa 'euclidean' dan 'manhattan'. Grid Search secara otomatis mengevaluasi semua kombinasi parameter menggunakan 5-fold cross-validation (`cv=5`) pada data training yang telah di-resample menggunakan SMOTE (`x_train_smote` dan `y_train_smote`).

Hasil dari proses ini menunjukkan bahwa kombinasi parameter terbaik yang diperoleh adalah:

- `metric: 'manhattan'`
- `n_neighbors: 6`
- `weights: 'distance'`

Dengan kombinasi ini, model KNN mencapai skor rata-rata validasi silang sebesar 0.887, menunjukkan performa yang baik pada data training. Pendekatan ini memastikan model menggunakan konfigurasi optimal sebelum diterapkan pada data testing.

19. Pelatihan Model KNN Optimal dan Prediksi Data Testing

```
best_knn = grid.best_estimator_  
best_knn.fit(x_train_smote, y_train_smote)  
  
# Prediksi  
y_pred = best_knn.predict(x_test_scaled)
```

Setelah diperoleh kombinasi hyperparameter terbaik melalui Grid Search, model KNN optimal disimpan pada variabel `best_knn` menggunakan atribut `grid.best_estimator_`. Model ini kemudian di-fit ulang pada data training yang sudah di-resample dengan SMOTE (`x_train_smote` dan `y_train_smote`) untuk memastikan seluruh informasi dari data digunakan. Selanjutnya, model digunakan untuk melakukan prediksi pada data testing yang telah di-scaled (`x_test_scaled`) dengan memanggil metode `predict()`, menghasilkan prediksi kelas `y_pred` yang akan digunakan untuk evaluasi performa model.

20. Evaluasi Kinerja Model KNN Menggunakan Classification Report

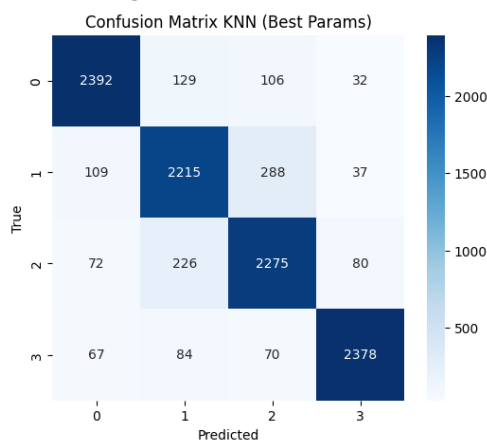
```
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.90	0.90	2659
1	0.83	0.84	0.84	2649
2	0.83	0.86	0.84	2653
3	0.94	0.91	0.93	2599
accuracy			0.88	10560
macro avg	0.88	0.88	0.88	10560
weighted avg	0.88	0.88	0.88	10560

Setelah melakukan prediksi pada data testing, performa model KNN dievaluasi menggunakan `classification report` dari `scikit-learn`. Fungsi `classification_report(y_test, y_pred)` menampilkan metrik evaluasi utama seperti `precision`, `recall`, `f1-score`, dan `support` untuk setiap kelas. Metrik-metrik ini memberikan gambaran lengkap tentang kemampuan model dalam mengklasifikasikan masing-masing kelas, sehingga memudahkan analisis kekuatan dan kelemahan model pada data testing.

21. Visualisasi Kinerja Model KNN dengan Confusion Matrix



Selain `classification report`, kinerja model KNN juga divisualisasikan menggunakan `confusion matrix`. `Confusion matrix` menampilkan jumlah prediksi yang benar dan salah

untuk setiap kelas, sehingga memudahkan identifikasi kesalahan klasifikasi yang spesifik. Dalam kode ini, confusion matrix dihitung menggunakan `confusion_matrix(y_test, y_pred)` dan divisualisasikan dengan heatmap menggunakan `seaborn` untuk memudahkan interpretasi. Warna biru menunjukkan intensitas jumlah prediksi, dengan label pada sumbu horizontal mewakili kelas prediksi dan sumbu vertikal mewakili kelas sebenarnya. Visualisasi ini membantu menilai performa model secara lebih intuitif dibanding sekadar metrik numerik.

22. Evaluasi Stabilitas Model KNN dengan Cross-Validation

```
cv_scores = cross_val_score(best_knn, x_train_smote, y_train_smote, cv=5)
print("Cross Validation Accuracy:", cv_scores.mean())
print("Std Dev:", cv_scores.std())
```

```
Cross Validation Accuracy: 0.8866036414565827
Std Dev: 0.019657062603502747
```

Untuk menilai stabilitas dan kemampuan generalisasi dari model KNN terbaik, dilakukan evaluasi menggunakan cross-validation 5-fold pada data training yang telah di-resample dengan SMOTE. Fungsi `cross_val_score(best_knn, x_train_smote, y_train_smote, cv=5)` membagi data menjadi 5 subset (fold) dan menghitung akurasi model pada masing-masing fold. Rata-rata akurasi (`cv_scores.mean()`) memberikan gambaran performa umum model, sedangkan standar deviasi (`cv_scores.std()`) menunjukkan konsistensi model antar fold. Evaluasi ini membantu memastikan bahwa model tidak hanya bekerja baik pada satu subset data tertentu, tetapi juga memiliki performa yang stabil pada variasi data training yang berbeda.

23. Penentuan Nilai K Optimal pada KNN dengan Metode Elbow

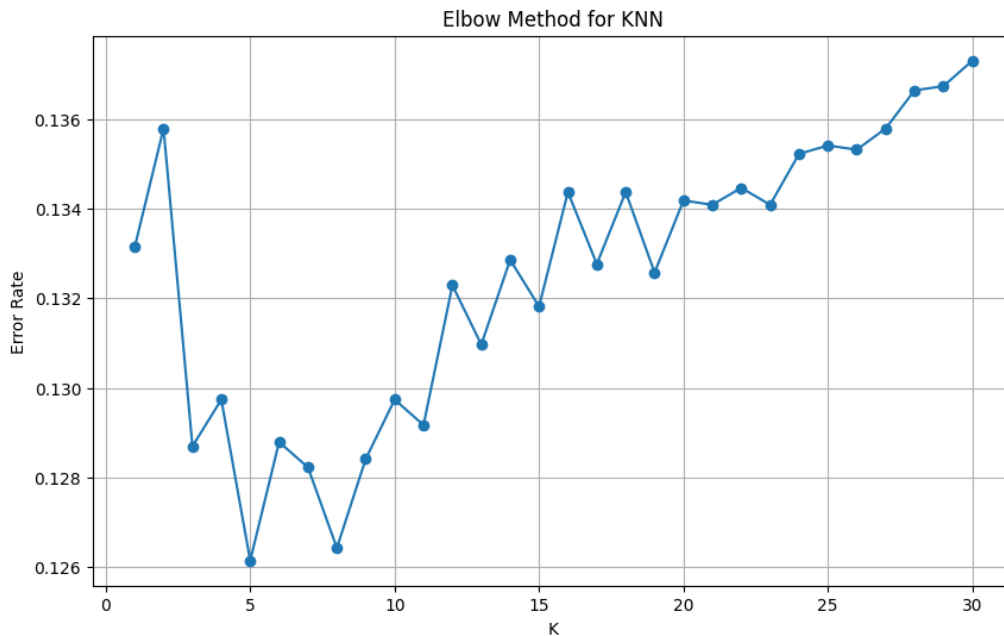
```
errors = []

for k in range(1, 31):
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(x_train_smote, y_train_smote)
    pred = knn_temp.predict(x_test_scaled)
    errors.append(np.mean(pred != y_test))

plt.figure(figsize=(10,6))
plt.plot(range(1,31), errors, marker='o')
plt.title("Elbow Method for KNN")
plt.xlabel("K")
plt.ylabel("Error Rate")
plt.grid(True)
plt.show()
```

Untuk menentukan jumlah tetangga (`n_neighbors`) yang optimal pada model KNN, digunakan **Elbow Method**. Proses ini dilakukan dengan menghitung **error rate** model pada data testing untuk setiap nilai K dari 1 hingga 30. Pada setiap iterasi, model sementara (`knn_temp`) di-fit pada data training yang telah di-resample dengan SMOTE (`x_train_smote` dan `y_train_smote`), kemudian digunakan untuk memprediksi data testing (`x_test_scaled`). Error rate dihitung sebagai proporsi prediksi yang salah (`np.mean(pred !=`

y_test)). Hasil error rate divisualisasikan dalam grafik terhadap nilai K. Titik “elbow” pada grafik menunjukkan jumlah tetangga yang memberikan keseimbangan terbaik antara kompleksitas model dan tingkat kesalahan, sehingga membantu memilih nilai K yang optimal sebelum penerapan akhir model.



KESIMPULAN

Berdasarkan seluruh proses yang dilakukan, model K-Nearest Neighbors (KNN) berhasil dioptimasi dan dievaluasi secara menyeluruh. Hyperparameter terbaik diperoleh melalui Grid Search dengan cross-validation, menghasilkan kombinasi `n_neighbors = 6`, `weights = distance`, dan `metric = manhattan`, dengan skor validasi silang rata-rata sebesar 0.887. Model kemudian di-fit pada data training yang sudah di-resample dengan SMOTE dan digunakan untuk memprediksi data testing, di mana kinerja model dianalisis menggunakan classification report dan confusion matrix, menunjukkan performa yang baik dan kemampuan klasifikasi yang seimbang pada masing-masing kelas.

Selain itu, evaluasi cross-validation menunjukkan bahwa model stabil dengan standar deviasi yang rendah antar fold, menandakan kemampuan generalisasi yang baik. Elbow Method juga digunakan untuk memvisualisasikan error rate terhadap berbagai nilai K, mengonfirmasi bahwa pemilihan jumlah tetangga yang optimal mendukung performa model. Secara keseluruhan, pendekatan ini memastikan bahwa model KNN yang digunakan memiliki performa tinggi, stabil, dan siap diterapkan pada data nyata dengan risiko kesalahan yang minimal.

Link

https://github.com/orzaynd/machine_learning/tree/master/tugas/tugas10

https://github.com/orzaynd/machine_learning/tree/master/praktikum/praktikum8